LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Advanced Topics in Parallel Computing

Dr. Karl Fürlinger
Lehrstuhl für Kommunikationssysteme und Systemprogrammierung

Spinlocks for Mutual Exclusion



Overview (1)

- Our setting:
  - We have a parallel shared memory system
  - We want to provide a mechanism for **mutual exclusion**, aka. a "**lock**" or "**mutex**"
  - Mutual exclusion means **only one thread at a time** may execute a block of code
  - That block of code is called a **critical section (CS)**

- Goals
  - Typically, we need mutual exclusion because we want to **protect a shared resource** from concurrent access, most often variables in shared memory
  - In our setting, the thread holding the lock has **exclusive (read/write) access**
  - Later: **reader-writer locks**, other generalizations

Overview (2)

- Spinning vs. Blocking
  - In our setting, threads waiting to acquire locks just **spin**, repeatedly reading some variable in memory and waiting for it to change – i.e., they are "**busy waiting**". Another option is to relinquish control to a scheduler, also called blocking – more later
  - Busy waiting doesn't work well if #threads > #cores, but may be a good choice when we have exclusive control over resources (HPC, embedded systems)
  - Synchronization in the OS scheduler, in turn, also relies on these mechanisms

Overview (3)

- Platform:
  - The system may have a **relaxed hardware memory model** (non-SC)
  - We can make use of **synchronizing instructions**, e.g., synchronizing loads/stores and fences, to prohibit reorderings
  - Additionally, **read-modify-write** operations are available on modern systems (e.g., compare-and-swap CAS, fetch-and-add FAA, etc.)
  - In C++ we can express all of the above using **operations on atomic types** (std::atomic<T>)

- Pseudocode in the following slides:
  - **.store()**, **.load()** etc. are **synchronizing instructions**, with optional reordering limitation specifications, e.g., **val.load(R||RW)**
  - There is a **total global order of all synchronizing instructions**
  - Other ("regular") instructions can be reordered as allowed by reordering limits

Historical Timeline for Spinlocks

- Early algorithms (1960s-1980s):
  - (Using **atomic load and store operations only**)
  - **Peterson's** algorithm (2-thread solution)
  - **Filter lock** (Peterson's algorithm for N threads)
  - **Tournament lock** (Peterson's algorithm with a tournament tree)
  - Lamport's **bakery algorithm** (N-thread solution)

- More recent algorithms (since the 1980s):
  - (Using **read-modify-write operations**)
  - Test-and-set **(TAS) lock**, Test and test-and-set **(TTAS) lock**
  - **Ticket lock**
  - Queue-based locks e.g., **MCS lock, CLH lock**

Event Model (Assuming SC Memory)



- We'll work with a simple event model in the following
  - Threads are state machines, state transitions are **events**
  - Events are **instantaneous**
  - There is a **total order on events**, for different events f, g either f->g or g->f

- Two events f->g define an interval I=(f,g)
  - There is a **partial order on intervals**:
  - If I=(f,g) and J=(k,l) then **I->J** iff. g->k
  - Intervals not related by the -> relation are said to be **concurrent**
  - Example for intervals: **critical sections (CS)**

- In a program with loops a single statement can give rise to many events, so we denote the j-th instance of an event $\mathbf{f^j}$
  - Similarly, we denote the j-th occurrence of an interval with $\mathbf{I^j}$

Desiderata for Locks

- **Mutual Exclusion**
  - Critical sections do not overlap. For threads A and B (A≠B) and integers j and k

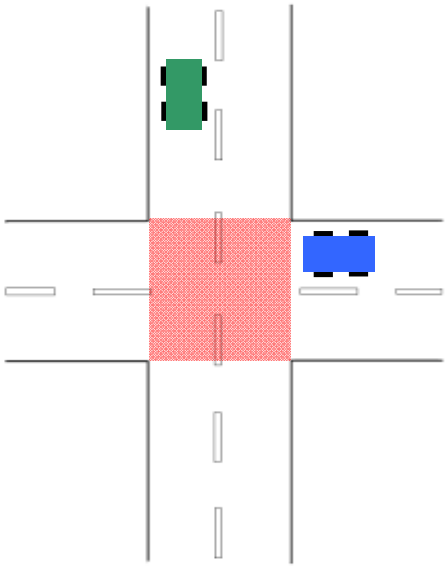
    $\mathbf{CS_A^k}$ -> $\mathbf{CS_B^j}$ or $\mathbf{CS_B^j}$ -> $\mathbf{CS_A^k}$

- **Deadlock Freedom**
  - If one or more threads try to acquire the lock, then some thread will succeed. An individual thread may be stuck trying to acquire the lock forever (a situation called **starvation**), but if this is the case, other threads are completing an infinite number of critical sections.

- **Starvation Freedom**
  - Every thread that tries to acquire the lock eventually succeeds
  - Thus, starvation freedom implies deadlock freedom

- **Fairness**
  - If thread A tries to acquire the lock before thread B, then A will enter the CS before B

Cars and Intersection Analogy



- **Mutual Exclusion**: Only one car at a time in the critical section (red area)
- **Deadlock freedom**: if one or more cars want to enter the intersection, some car will. An individual car may be stuck forever, but only if cross-traffic continues to flow forever
- **Starvation freedom**: Each car that arrives at the intersection will eventually enter it
- **Fairness**: Cars enter the intersection in the order they arrive at the intersection

Deadlock / Starvation / Fairness

- These terms apply to any kind of “service” that may be used by multiple threads, or “agents” in general
  - For locks the “service” is mutual exclusion
  - Similar considerations hold for concurrent data structures, e.g., a concurrent queue
  - We’ll discuss these **progress conditions** in more detail later

- Of course, **using multiple locks** may lead to deadlock if one is not careful, even if the lock implementation is guaranteed to be deadlock-free:

```
Lock l1, l2;

// Thread 0                  // Thread 1
l1.acquire();                l2.acquire();
  l2.acquire();                l1.acquire()
  l2.release();                l1.release()
l1.release()                 l2.release()
```

Peterson's Algorithm (Pseudo code)

```
class PetersonLock {
  (0, 1) turn;
  bool interested[0..1] = {false, false};

acquire():
  other = 1 – self;
  interested[self] = true;    // I'm interested
  turn = other;               // you can go first

  while (interested[other] && turn==other) {} // spin

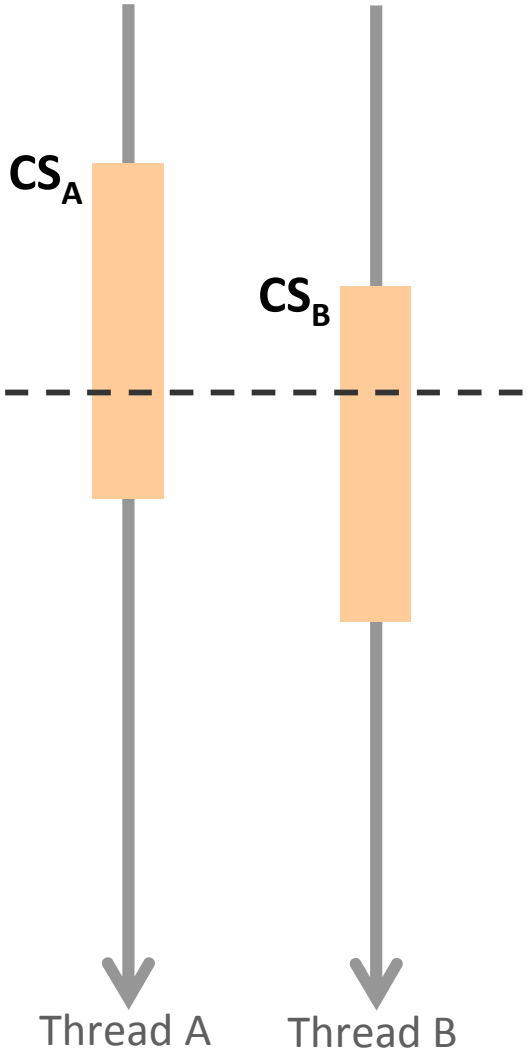
release():
  interested[self] = false;
}
```

Assume Sequential Consistency for simplicity, i.e., every access is a synchronizing access.

- Discussion
  - **self** is the executing thread's ID (0 or 1)
  - Boolean array **interested** to indicate desire to acquire lock
  - Variable **turn** (0 or 1) is used to break the tie (when both threads are interested)

The Peterson Lock Provides Mutual Exclusion

- Proof by contradiction:
  - Assume overlapping critical sections:



Thread A and B have entered their CS:

$write_A(interested[A]=true)$ -> $write_A(turn=B)$ -> $read_A(interested[B])$ -> ($read_A(turn)$) -> **CS_A**

$write_B(interested[B]=true)$ -> $write_B(turn=A)$ -> $read_B(interested[A])$ -> ($read_B(turn)$) -> **CS_B**

Now assume w/o loss of generality

$write_A(turn=B)$ -> $write_B(turn=A)$

Since there are no other accesses to interested[] until release():

$write_A(interested[A]=true)$ -> $write_A(turn=B)$ -> $write_B(turn=A)$ -> $read_B(interested[A])$==**true**

But B has entered the CS! It follows that

$read_B(turn)$==**B**

Contradiction!

The Peterson Lock is Starvation Free

- Proof by contradiction
  - Assume w.o.l.g. thread A is stuck forever, not acquiring the lock
  - It must be stuck in

```
while (interested[B] && turn==B) {} // spin
```

- What is thread B doing in the mean time?
  - interested[B] is true, so B must have called acquire() but not yet release()
  - So B must be stuck forever too, executing

```
while (interested[A] && turn==A) {} // spin
```

  But that would mean that turn==B while also turn==A, a **contradiction**.

- Consequence: The Peterson Lock is deadlock-free!

- What about fairness?

Filter and Tournament Locks

- Peterson lock is a two-thread solution only
  - **Filter** and **tournament** locks extend the idea to N threads

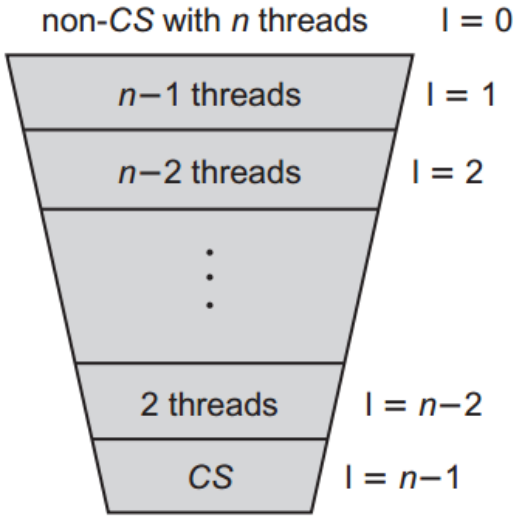
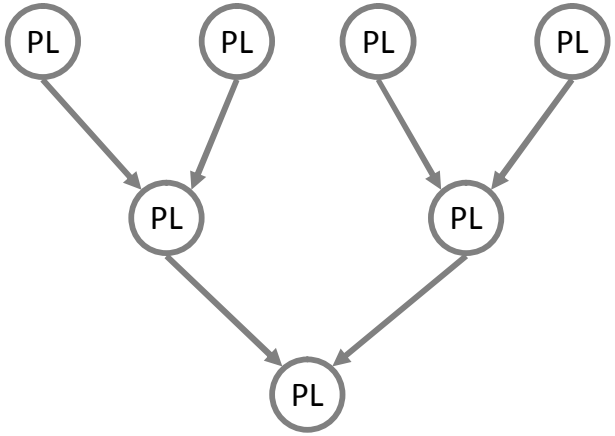


Image source: Maurice Herlihy, Nir Shavit
"The Art of Multiprocessor Programming"



- Filter lock
  - Starvation free
  - $O(N^2)$ time/mem access
  - O(N) space
- Tournament lock
  - Starvation free
  - O(log N) time/mem access
  - O(N) space

Discussion

- It can be shown that any deadlock-free mutual exclusion algorithm using **only reads and writes** requires O(N) space, where N is the number of threads
  - This motivates the usage of **stronger synchronization primitives** (later)

- Some of the locks discussed so far are starvation-free but not **fair**
  - A thread could be bypassed many times before acquiring the lock
  - This motivates the usage of **queues** for managing waiting (later)

Lamport's Bakery Algorithm

- Lamport's **Bakery algorithm** addresses **fairness**
  - Idea: Works like the number dispensing systems found in some shops. When calling acquire(), each thread "picks a number" and then waits until there is no other thread with a smaller number
  - "Picking a number" in the bakery algorithm is distributed: to pick a number, each threads reads all other thread's chosen numbers, computes the **max** and uses **max+1** as it's own number
  - A **ticket lock** works in a similar way but uses a central atomic counter to pick the next number (using fetch-and-add, more later)

Bakery Algorithm

```
class BakeryLock {
  bool interested[] = {false,...}; // indexed by thread ID
  int ticket[] = {0,...};          // indexed by thread ID

acquire():
  interested[self] = true;     // I'm interested
  int myticket = ticket.max() + 1;
  ticket[self] = myticket;

  for_each (Thread i)
     while( interested[i] &&
            <ticket[i],i> < <myticket,self> ) {}  // spin

release():
  interested[self] = false;
};
```

For simplicity, let's assume SC

Comparison of <number,thread-id> pairs, for the case where two threads choose the same number

- **Bakery Lock**
  - Provides deadlock freedom, starvation freedom and fairness

Test-and-Set (TAS) Lock

```
class TASLock {
  bool f = false;

acquire():
  while TAS(&f) {} // spin
  fence(R||RW);

release():
  f.store(false, RW||);
}
```

- TAS: atomically set to **true** and return previous value
  - If TAS returns **false**, this indicates that the lock was available (and is now acquired)
  - If TAS returns **true**, this indicates that the lock was not available -> spin

- Problems with the TAS lock
  - All threads spins on the **same variable** and repeatedly apply a RMW operation (TAS)
  - The variable will be in all participating thread's caches
  - Write permission is needed for the RMW operation, so several caches will repeatedly "fight" over getting exclusive write access
  - I.e., there will be a lot of coherence traffic

Test and Test-and-Set (TTAS) Lock

```
class TTASLock {
  bool f = false;

acquire():
  while TAS(&f) {
    while f {} // spin
  }
  fence(R||RW);

release():
  f.store(false, RW||);
}
```

- Instead of spinning on the result of TAS, read the value directly in an inner loop
- Then use TAS and try to obtain the lock
- More efficient, because the read in the inner loop can be satisfied from the local cache as long as the value hasn't changed (the lock is not released)

Ticket Lock

Ticket Lock

- Same idea as bakery lock, but using a central counter and FAI
- Provides deadlock freedom, starvation freedom and fairness

```
class TicketLock {
  int next_ticket = 0;
  int now_serving = 0;

acquire():
  int my_ticket = FAI(&next_ticket);

  while(true) {
    ns = now_serving.load();
    if( ns == my_ticket ) break;
  }
  fence(R||RW);

release():
  int tmp = now_serving + 1;
  now serving.store(tmp, RW||);
}
```

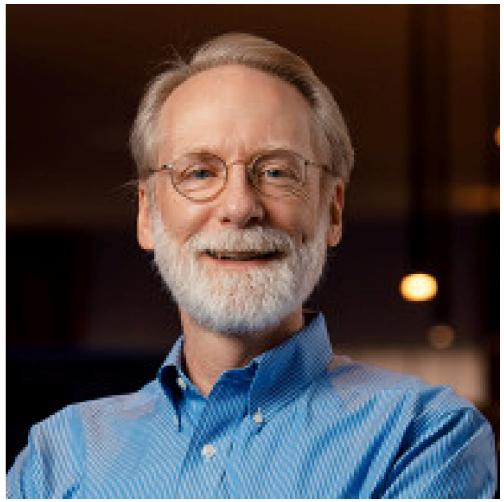
MCS Lock (1)

- Problems with the locks so far:
  - Often no fairness guarantees
  - Spinning on a centralized location (performance)

- Ideas:
  - Have each waiting thread spin on a separate location
  - Maintain fairness by linking these locations in order of arrival



John Mellor-Crummey, Rice University



Michael L. Scott, Univ. of Rochester

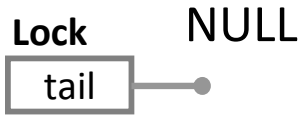
MCS Lock (2)

- MCS lock is represented by a **queue** (singly linked list of nodes)
  - A queue node consists of a **pointer** to the next node (or NULL) and a **flag**
  - A thread that wants to **acquire** the lock adds a node to the end of the queue and spins on it's own flag
  - A thread that **releases** the lock sets the flag of the successor node (if any), thus permitting the other thread to acquire the lock
  - Requires RMW operations: **compare-and-swap** and **swap**
  - The lock is represented by a pointer to the **tail of the queue**

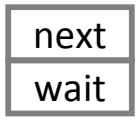
- Advantages:
  - Fair: locks are acquired in the order of arrival
  - Performance: No spinning on a central flag, each thread spins on it's own local flag

MCS Design and Intuition (1)

Initial Lock:

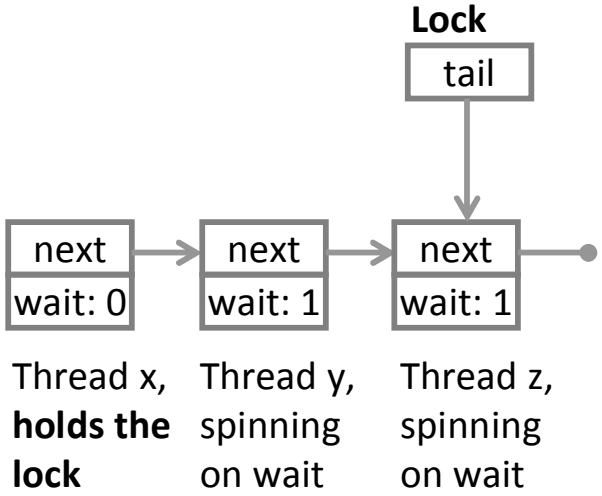


qnode Record:



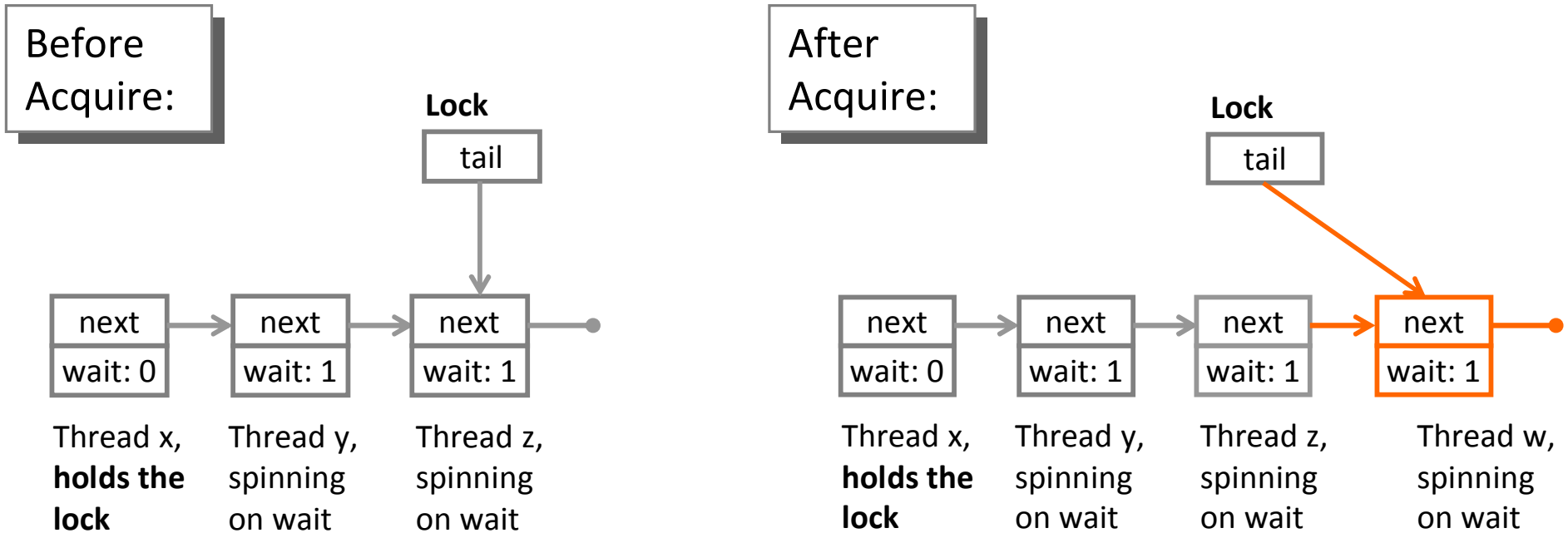
- *next* points to next qnode
- *wait* is a Boolean flag the thread will spin on

Situation in which several threads wait to acquire the lock:



- *tail* always points to end of queue
- Each thread spins on its own flag

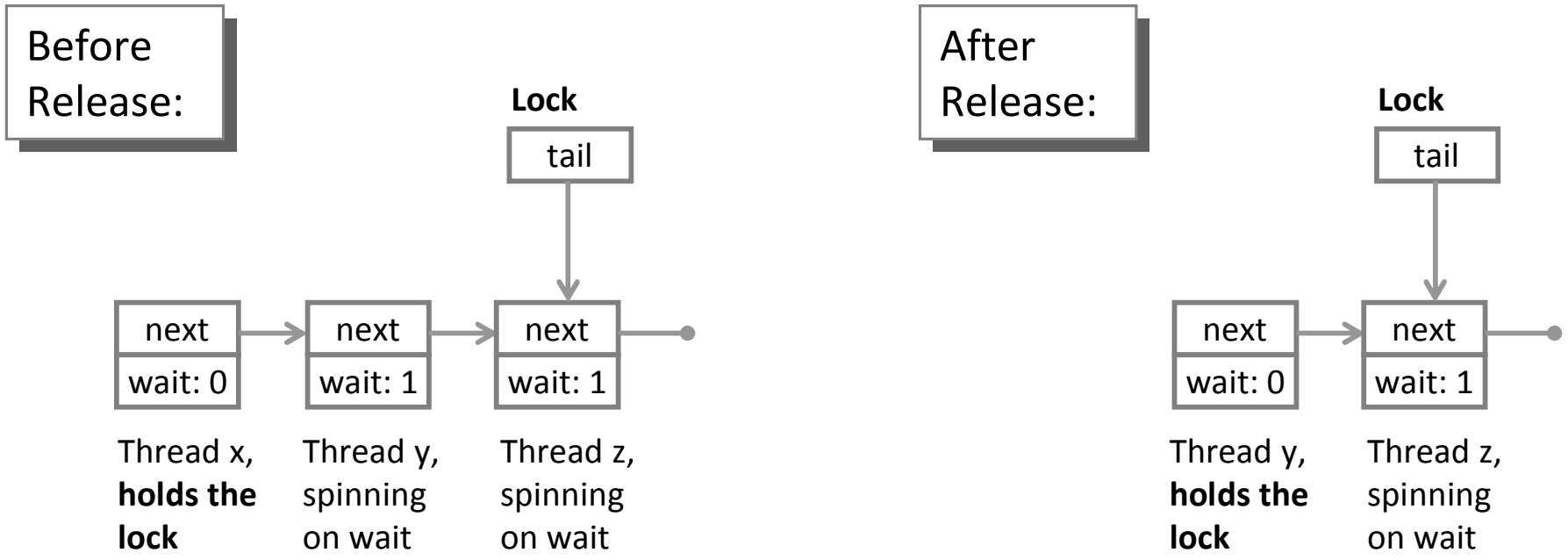
MCS Acquire Operation




```
word swap(word *a, word new):
atomic { word old = *a;
         *a = new;
         return old; }
```

- Acquire:
  - 1. Atomically swing tail pointer to new element (using **swap**)
  - 2. Let the next pointer of the previous tail point to the new element

MCS Release Operation (1)

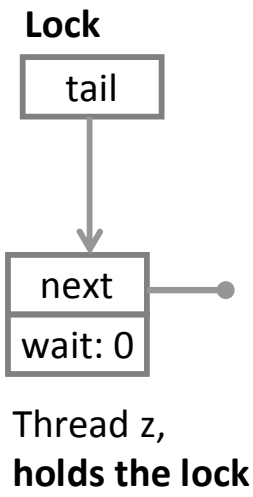


- Release (simple case: the node has a successor):
  - Follow the next pointer and set the wait flag to of the successor to 0

MCS Release Operation (2)

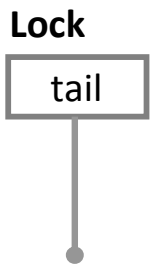
- Release (more complex case: the node has no successor):
  - Subcase A) No intervening acquire

Before Release:



w/o intervening acquire –
CAS(&tail, p, NULL) successful

After Release:

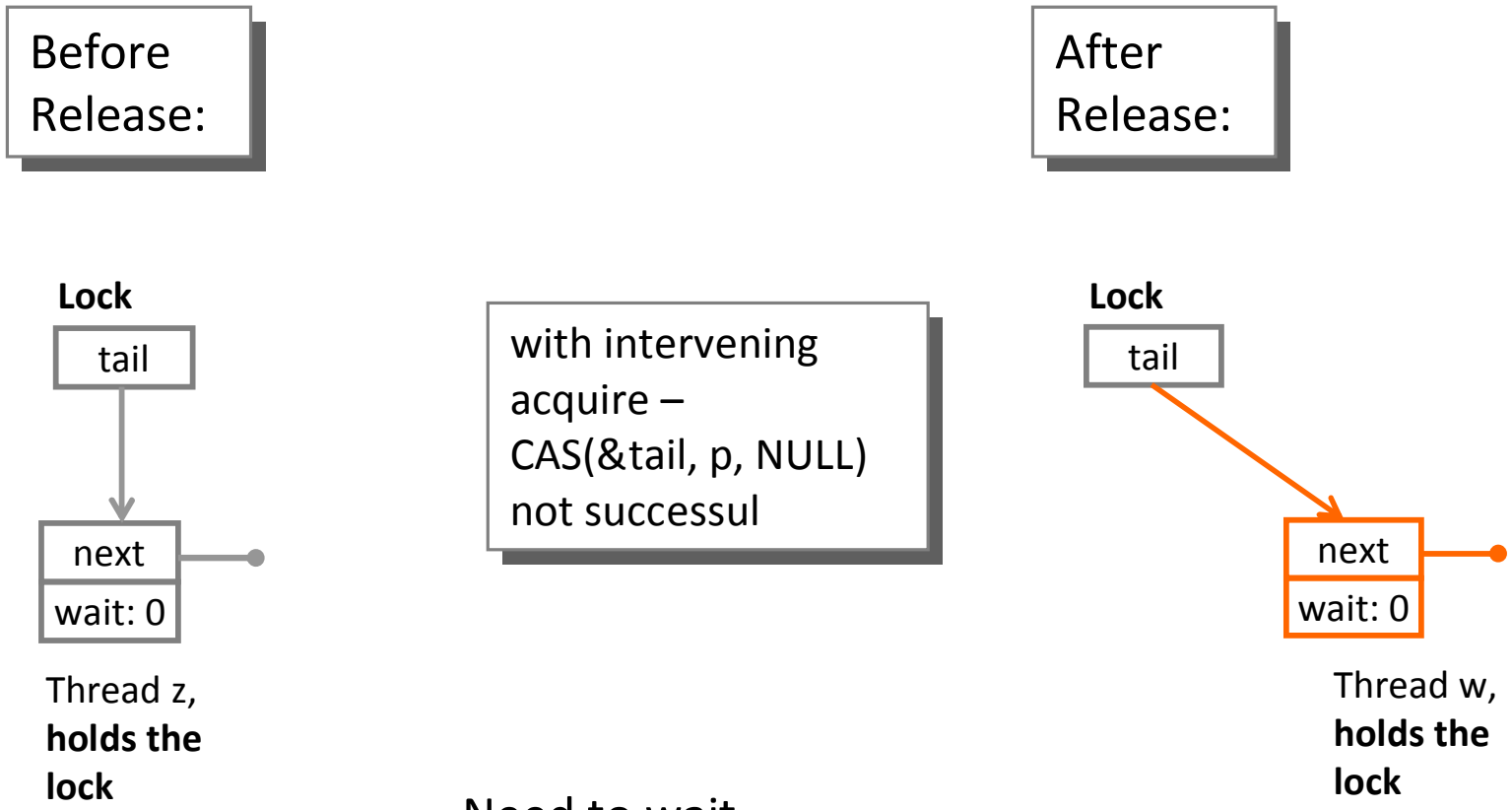


Definition of CAS:

```
bool CAS(word *a, word expected, word desired):
atomic { bool t = (*a == expected);
         if (t) *a = desired;
         return t; }
```

MCS Release Operation (3)

- Release (more complex case: the node has no successor):
  - Subcase B) Intervening acquire



MCS Lock Pseudo code

```
type qnode {
  qnode* next;  // next node in the queue
  bool wait;    // flag to spin on
}
class lock {
  qnode* tail = NULL;

acquire(qnode* p):
  p->next = NULL;
  p->wait = true;
  qnode* prev = swap(&tail, p, W||);
  if prev != NULL          // queue was nonempty
    prev->next.store(p);  // link in new node
    while p->wait.load(); // spin on flag
  fence(R||RW);

release(qnode* p):
  qnode* succ = p->next.load(RW||);
  if succ == NULL              // no known successor
    if CAS(&tail, p, NULL) return;
    repeat succ = p->next.load() until succ != NULL
  succ->wait.store(false);
}
```

MCS Lock Discussion

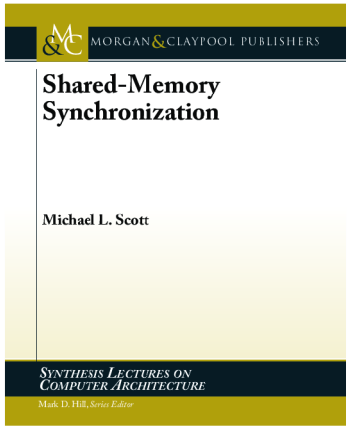
- MCS is a frequently used lock that performs well in practice
  - FIFO guarantee and spinning on distinct locations

- Note the difference in the lock interface:
  - acquire() and release() expect to be provided with a **pointer to a qnode**
  - The qnode is typically allocated in the stack frame of the thread
  - For interchangability with other locks, this is undesirable
  - There is a "**K42"** variant of the MCS lock, which provides the standard lock interface; K42: past IBM OS project

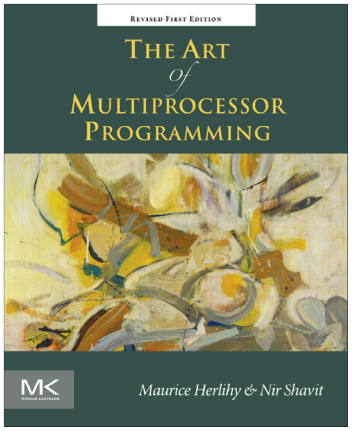
Summary

- Load-store only algorithms are primarily of interest in the theory community today
- The test-and-set lock and variants can work well for small numbers of cores and low contention
  - Exponential backoff can be used in presence of contention
- Queue-based locks (e.g., MCS) are scalable and work better in case of contention
  - MCS lock is used in many operating systems

Further Reading



- Michael L. Scott *"Shared-memory synchronization"* Synthesis Lectures on Computer Architecture (2013)
  - Chapter 4 “Practical Spin Locks”



- Maurice Herlihy, Nir Shavit *"The Art of Multiprocessor Programming"*
  - Chapter 2 “Mutual Exclusion”
  - Chapter 7 “Spin Locks and Contention”